
Grafos (Graphs)

Estrutura de Dados - SENAC BCC

Conteudo

1. O que e um Grafo?
2. Matriz de Adjacencias
3. Lista de Adjacencias
4. Quando usar cada representacao?
5. Busca em Profundidade (DFS)
6. Busca em Largura (BFS)
7. DFS vs BFS
8. Casos de uso no mundo real

1. O que é um Grafo?

Um **grafo** é uma estrutura de dados que modela **relacionamentos** entre elementos. É composto por:

- **Vertices** (ou nós): os elementos
- **Arestas** (ou arcos): as conexões entre os elementos

Analogia: Pense em um mapa de cidades conectadas por estradas. Cada cidade é um vértice e cada estrada é uma aresta. Você pode perguntar: "existe estrada de A para B?" ou "qual o caminho mais curto de A até C?".

Tipos de Grafos

Tipo	Descrição	Exemplo
Não direcionado	Arestas nos dois sentidos	Amizade no Facebook
Direcionado (digrafo)	Arestas com direção ($A \rightarrow B \neq B \rightarrow A$)	Seguir no Instagram
Ponderado	Arestas têm peso/custo	Distância entre cidades
Não ponderado	Todas arestas mesmo peso	Rede de contatos

Terminologia

- **Grau** de um vértice: número de arestas conectadas a ele
- **Caminho**: sequência de vértices conectados por arestas
- **Ciclo**: caminho que começa e termina no mesmo vértice
- **Grafo conexo**: existe caminho entre qualquer par de vértices
- **Adjacência**: dois vértices são adjacentes se existe aresta entre eles

2. Matriz de Adjacencias

Uma **matriz quadrada** de tamanho $V \times V$ (onde V = numero de vertices). A posicao $[i][j]$ indica se existe aresta do vertice i para o vertice j .

Grafo:	Matriz de Adjacencias:
0 --- 1	0 1 2 3 4

	0 0 1 1 0 0
2 --- 3	1 1 0 0 1 0
	2 1 0 0 1 0
4	3 0 1 1 0 1
	4 0 0 0 1 0

Implementacao em C#

```
public class GrafoMatriz
{
    private int[,] matriz;
    private int vertices;

    public GrafoMatriz(int vertices)
    {
        this.vertices = vertices;
        this.matriz = new int[vertices, vertices];
    }

    public void AdicionarAresta(int origem, int destino)
    {
        matriz[origem, destino] = 1;
        matriz[destino, origem] = 1; // nao direcionado
    }

    public bool ExisteAresta(int origem, int destino)
    {
        return matriz[origem, destino] == 1;
    }
}
```

Complexidade

Operacao	Complexidade
Verificar se existe aresta	$O(1)$
Adicionar aresta	$O(1)$
Remover aresta	$O(1)$
Listar vizinhos de um vertice	$O(V)$
Espaco em memoria	$O(V^2)$

- **Vantagem:** Acesso direto $O(1)$ para verificar se existe aresta
- **Desvantagem:** Usa $O(V^2)$ de memoria mesmo com poucas arestas (grafo esparso)

3. Lista de Adjacencias

Cada vertice mantem uma **lista** com seus vizinhos. Usa um array de listas.

Grafo:	Lista de Adjacencias:
0 --- 1	0: [1, 2]
	1: [0, 3]
	2: [0, 3, 4]
2 --- 3	3: [1, 2, 4]
	4: [2, 3]
4	

Implementacao em C#

```
public class GrafoLista
{
    private List<int>[] adjacencias;
    private int vertices;

    public GrafoLista(int vertices)
    {
        this.vertices = vertices;
        this.adjacencias = new List<int>[vertices];
        for (int i = 0; i < vertices; i++)
            adjacencias[i] = new List<int>();
    }

    public void AdicionarAresta(int origem, int destino)
    {
        adjacencias[origem].Add(destino);
        adjacencias[destino].Add(origem);
    }

    public List<int> Vizinhos(int vertice)
    {
        return adjacencias[vertice];
    }
}
```

Complexidade

Operacao	Complexidade
Verificar se existe aresta	O(grau do vertice)
Adicionar aresta	O(1)
Remover aresta	O(grau do vertice)
Listar vizinhos	O(grau do vertice)
Espaco em memoria	O(V + E)

- **Vantagem:** Usa menos memoria para grafos esparsos: $O(V + E)$
- **Desvantagem:** Verificar se existe aresta pode custar $O(\text{grau})$

4. Quando usar cada representacao?

Criterio	Matriz	Lista
Grafo denso (muitas arestas)	Melhor	-
Grafo esparso (poucas arestas)	-	Melhor
Verificar arestas com frequencia	Melhor	-
Listar vizinhos com frequencia	-	Melhor
Memoria limitada	-	Melhor

Metodo Vizinhos: comparando as duas representacoes

O metodo Vizinhos retorna todos os vertices adjacentes a um vertice dado. A implementacao e bem diferente em cada representacao, e ilustra o trade-off fundamental entre elas.

Matriz de Adjacencias

Precisa percorrer **toda a linha** da matriz para encontrar os vizinhos. Mesmo que o vertice tenha poucos vizinhos, sempre percorre V posicoes.

```
public List<int> Vizinhos(int vertice)
{
    List<int> vizinhos = new List<int>();
    for (int i = 0; i < vertices; i++)
    {
        if (matriz[vertice, i] == 1)
        {
            vizinhos.Add(i);
        }
    }
    return vizinhos;
}
```

Complexidade: $O(V)$ — sempre percorre todas as colunas, independente do numero de vizinhos.

Lista de Adjacencias

Ja tem a lista pronta — basta retornar. Acesso direto, sem necessidade de busca.

```
public List<int> Vizinhos(int vertice)
{
    return adjacencias[vertice];
}
```

Complexidade: $O(1)$ — retorna a referencia da lista diretamente. Iterar sobre os vizinhos custa $O(\text{grau do vertice})$.

Por que isso importa?

Os algoritmos DFS e BFS chamam Vizinhos() para **cada vertice** visitado. Essa diferenca de implementacao e o que faz a complexidade mudar:

Algoritmo	Com Matriz	Com Lista
Vizinhos (1 vertice)	$O(V)$	$O(\text{grau})$
DFS / BFS (todos)	$O(V^2)$	$O(V + E)$

5. Busca em Profundidade (DFS)

A DFS explora o grafo indo o **mais fundo possivel** antes de retroceder. Funciona como explorar um labirinto: voce segue por um corredor ate chegar a um beco sem saida, depois volta e tenta outro caminho.

Como funciona

- Comece em um vertice, marque como visitado
- Visite um vizinho nao visitado e repita
- Se todos os vizinhos ja foram visitados, retroceda (backtrack)
- Repita ate visitar todos os vertices alcancaveis

Visualizacao

Grafo: Ordem de visitacao DFS (inicio: 0):

```
0 --- 1                      0 -> 1 -> 3 -> 2 -> 4
|                             
|                              Pilha (implicita na recursao):
2 --- 3                      [0] -> [0,1] -> [0,1,3] -> [0,1,3,2] -> [0,1,3,2,4]
|
4
```

Implementacao recursiva em C#

```
public void DFS(int vertice, bool[] visitados)
{
    visitados[vertice] = true;
    Console.Write(vertice + " ");

    foreach (int vizinho in Vizinhos(vertice))
    {
        if (!visitados[vizinho])
        {
            DFS(vizinho, visitados);
        }
    }
}
```

Complexidade da DFS

	Matriz de Adjacencias	Lista de Adjacencias
Tempo	$O(V^2)$	$O(V + E)$
Espaco	$O(V)$	$O(V)$

Aplicacoes da DFS

- Detectar ciclos em grafos
- Encontrar componentes conectados
- Ordenacao topologica (grafos direcionados)
- Resolver labirintos

6. Busca em Largura (BFS)

A BFS explora o grafo **nível por nível**, visitando todos os vizinhos de um vertice antes de avançar. Funciona como ondas se propagando em um lago quando voce joga uma pedra.

Como funciona

- Comece em um vertice, marque como visitado, coloque na **fila**
- Retire um vertice da fila
- Visite todos os vizinhos nao visitados e coloque-os na fila
- Repita ate a fila ficar vazia

Visualizacao

Grafo:	Ordem de visitacao BFS (inicio: 0):
0 --- 1	Nivel 0: 0
	Nivel 1: 1, 2
	Nivel 2: 3
2 --- 3	Nivel 3: 4
4	Fila: [0] -> [1,2] -> [2,3] -> [3,4] -> [4] -> []

Implementacao em C#

```
public void BFS(int inicio)
{
    bool[] visitados = new bool[vertices];
    Queue<int> fila = new Queue<int>();

    visitados[inicio] = true;
    fila.Enqueue(inicio);

    while (fila.Count > 0)
    {
        int vertice = fila.Dequeue();
        Console.Write(vertice + " ");

        foreach (int vizinho in Vizinhos(vertice))
        {
            if (!visitados[vizinho])
            {
                visitados[vizinho] = true;
            }
        }
    }
}
```



```
        fila.Enqueue(vizinho);
    }
}
}
```

Complexidade da BFS

	Matriz de Adyacencias	Lista de Adyacencias
Tempo	$O(V^2)$	$O(V + E)$
Espaco	$O(V)$	$O(V)$

Aplicacoes da BFS

- **Caminho mais curto** em grafos nao ponderados
- Encontrar componentes conectados
- Crawler de web (visitar paginas por nivel)
- Redes sociais (sugestao de amigos por grau de separacao)

7. DFS vs BFS

Critério	DFS	BFS
Estrutura auxiliar	Pilha (recursao)	Fila
Estrategia	Vai fundo, depois volta	Explora nivel a nivel
Caminho mais curto?	Nao garante	Sim (nao ponderados)
Uso de memoria	$O(V)$ pior caso	$O(V)$ pior caso
Melhor para...	Ciclos, backtracking	Caminho curto, niveis

Resumo: Use DFS quando precisar explorar todos os caminhos ou detectar ciclos. Use BFS quando precisar do caminho mais curto ou explorar por níveis de profundidade.

8. Casos de uso no mundo real

Uso	Exemplo
Redes sociais	Amizades, seguidores, sugestões
Navegação / GPS	Cidades e estradas, caminho mais curto
Internet	Roteamento de pacotes entre servidores
Compiladores	Dependências entre módulos
Jogos	Mapa de fases, pathfinding de NPCs
Biologia	Redes de interação de proteínas
Recomendação	Quem comprou X também comprou Y

Resumo: Representações

	Matriz de Adj.	Lista de Adj.
Espaco	$O(V^2)$	$O(V + E)$
Verificar aresta	$O(1)$	$O(\text{grau})$
Listar vizinhos	$O(V)$	$O(\text{grau})$
Adicionar aresta	$O(1)$	$O(1)$
Remover aresta	$O(1)$	$O(\text{grau})$
Melhor para	Grafos densos	Grafos esparsos